Advanced PHP

Amjahdi Walid
2334452
ISI S5

TP 6 : Namespaces en PHP

Exercice 1 — Éviter les collisions de noms

Les deux classes portent le même nom, mais leurs namespaces différents permettent de les utiliser ensemble.

ShopOrder.php

Classe Order du magasin (namespace App\Shop).

```php
<?php
namespace App\Shop;

class Order
{
    public function getTotal()
    {
        return "Total de la commande : 150€";
    }
}
```

AdminOrder.php

Classe Order de l'administration (namespace App\Admin).

```php
<?php
namespace App\Admin;

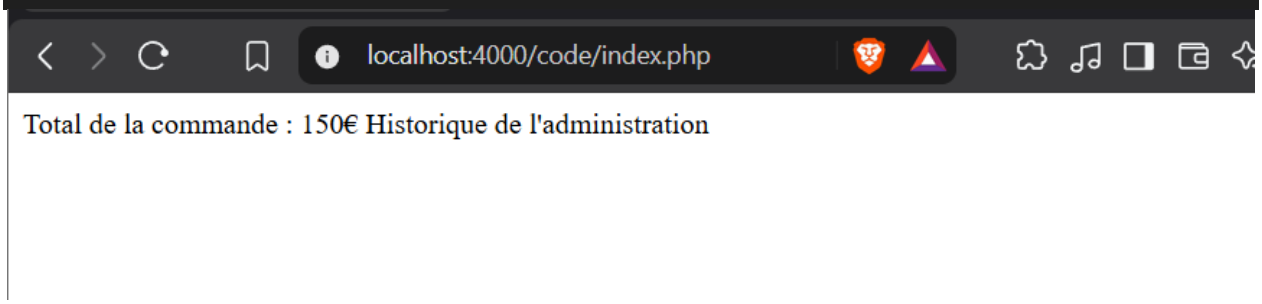
class Order
{
    public function getHistory()
    {
        return "Historique de l'administration";
    }
}
```

index.php

```php
<?php
require_once 'ShopOrder.php';
require_once 'AdminOrder.php';

$shopOrder = new \App\Shop\Order();
$adminOrder = new \App\Admin\Order();

echo $shopOrder->getTotal() . PHP_EOL;
echo $adminOrder->getHistory() . PHP_EOL;
```



Exercice 2 — Importation et alias

Les deux classes Logger sont importées. La seconde est renommée ExternalLogger avec as pour éviter le conflit.

Exercice2_Alias.php

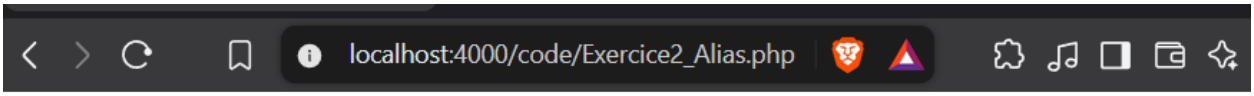
```php
<?php
namespace Core\Utils\Debug {
    class Logger {}
}

namespace Vendor\Log\External {
    class Logger {}
}

namespace App\Controller {
    use Core\Utils\Debug\Logger;
    use Vendor\Log\External\Logger as ExternalLogger;

    $localLogger = new Logger();
    $extLogger = new ExternalLogger();

    echo "Les deux loggers ont été instanciés avec succès !";
}
```



Les deux loggers ont été instanciés avec succès !

Exercice 3 — Le namespace global

L'erreur apparaît car DateTime sans préfixe est interprété comme App\Services\DateTime. La classe native DateTime se trouve dans le namespace global.

Méthode A : préfixer la classe native par un antislash (\DateTime).

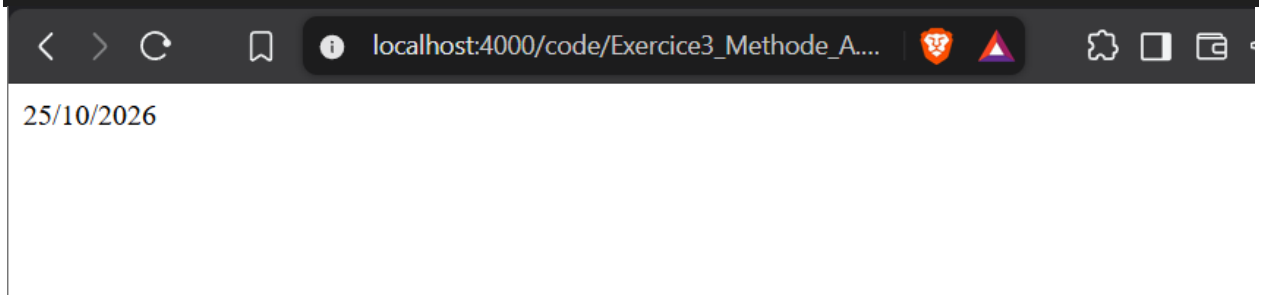
Exercice3_Methode_A.php

```php
<?php
namespace App\Services;

class DateFormat
{
    public function showDate()
    {
```

```php
        $date = new \DateTime();
        return $date->format('d/m/Y');
    }
}

$manager = new DateFormat();
echo $manager->showDate();
```



Exercice3_Methode_B.php

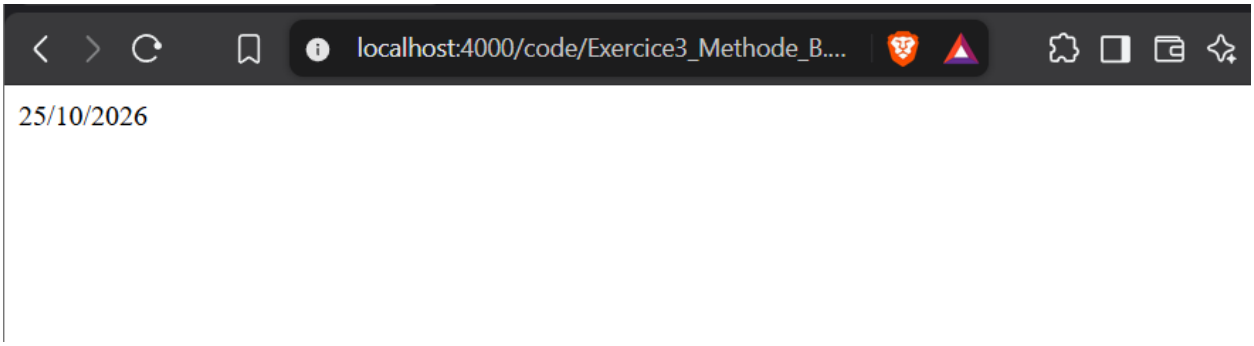
Méthode B : importer DateTime avec use DateTime; puis l'utiliser sans antislash.

```php
<?php
namespace App\Services;

use DateTime;

class DateFormat
{
    public function showDate()
    {
        $date = new DateTime();
        return $date->format('d/m/Y');
    }
}

$manager = new DateFormat();
echo $manager->showDate();
```



Quiz : Les namespaces en PHP

1. Où doit se situer obligatoirement l'instruction de déclaration d'un namespace dans un fichier PHP ?

A. N'importe où dans le fichier, du moment que c'est avant la première instanciation de classe.
B. Tout au début du fichier, immédiatement après la balise d'ouverture <?php, à l'exception des déclarations declare.
C. Juste avant la classe ou la fonction concernée par ce namespace.
D. À la toute fin du fichier PHP, juste avant la balise de fermeture ?>.

Réponse correcte : B

Explication : La déclaration du namespace doit être la première instruction du fichier, à l'exception d'une éventuelle instruction declare. Aucun code exécutable ni texte ne doit la précéder.

2. Que se passe-t-il si vous placez une instruction echo "Bonjour"; avant la déclaration namespace MonProjet; dans un fichier PHP ?

A. Le texte s'affiche normalement et le namespace s'applique au reste du fichier sans problème.
B. PHP ignore silencieusement l'affichage et applique le namespace.
C. Une erreur fatale est déclenchée, car la déclaration du namespace doit être la première instruction du fichier.
D. Cela transforme le namespace en un sous-namespace implicite.

Réponse correcte : C

Explication : Une instruction echo placée avant la déclaration du namespace provoque une erreur fatale. PHP exige que la déclaration du namespace précède les instructions exécutables.

3. Si vous êtes à l'intérieur du namespace MonProjet\Services, comment accédez-vous explicitement à la classe native globale DateTime de PHP sans instruction use ?

A. En écrivant \DateTime, avec un antislash au début.
B. En écrivant simplement DateTime, car PHP recherche toujours la classe dans le namespace global.
C. En écrivant global\DateTime.
D. En écrivant root:DateTime.

Réponse correcte : A

Explication : L'antislash initial indique que la classe appartient au namespace global. Ainsi, \DateTime désigne explicitement la classe native DateTime de PHP.

4. À l'intérieur du namespace MonApp\Controllers, vous écrivez $db = new Connection();. Comment PHP résout-il cette classe si aucun import use n'est spécifié ?

A. Il recherche uniquement la classe dans le namespace global, puis échoue si elle n'existe pas.
B. Il recherche la classe MonApp\Controllers\Connection. Si elle n'existe pas, PHP ne recherche pas automatiquement une classe Connection dans le namespace global.
C. Il génère immédiatement une erreur, car une instruction use est obligatoire pour toute instanciation de classe.
D. Il recherche dans tous les autres namespaces du projet de manière aléatoire.

Réponse correcte : B

Explication : Pour les classes, interfaces, traits et énumérations, un nom non qualifié est résolu relativement au namespace courant. PHP ne se rabat pas automatiquement sur le namespace global si la classe n'est pas trouvée. Pour utiliser une classe globale, on peut écrire new \Connection();.

5. Quelle est la syntaxe correcte pour importer la classe App\Entities\User dans un fichier appartenant à un autre namespace ?

A. include App\Entities\User;
B. import App\Entities\User;
C. use App\Entities\User;
D. namespace App\Entities\User;

Réponse correcte : C

Explication : Le mot-clé use permet d'importer un nom de classe dans le fichier afin de pouvoir l'utiliser sans répéter son nom complet. Il ne charge pas lui-même le fichier de la classe.

6. Comment importer la classe App\Http\Request en lui attribuant l'alias HttpRequest afin d'éviter un conflit de noms ?

A. use App\Http\Request rename HttpRequest;
B. use App\Http\Request as HttpRequest;
C. use HttpRequest = App\Http\Request;
D. use App\Http\Request(HttpRequest);

Réponse correcte : B

Explication : Le mot-clé as permet de définir un alias pour une classe importée. On peut ensuite utiliser HttpRequest à la place de son nom complet.

7. Vous devez utiliser deux classes nommées Logger, provenant respectivement de App\Log\Logger et de Vendor\Monolog\Logger. Que devez-vous faire ?

A. Importer les deux classes normalement : PHP remplacera automatiquement le premier import par le second.
B. Utiliser le mot-clé as pour attribuer un alias distinct à au moins l'une des deux classes.
C. Les préfixer obligatoirement par un double point dans le code, sans utiliser use.
D. Fusionner les deux namespaces en un seul bloc use sans alias.

Réponse correcte : B

Explication : Deux classes portant le même nom court ne peuvent pas être importées sous ce même nom dans le même espace de noms de compilation. Il faut utiliser des alias pour les distinguer.

8. Quelle est la syntaxe correcte d'un import groupé, disponible depuis PHP 7, permettant d'importer les classes Model, View et Controller du namespace App\Core ?

A. use App\Core\{Model, View, Controller};
B. use App\Core(Model, View, Controller);
C. use App\Core\Model, View, Controller;
D. group use App\Core { Model, View, Controller };

Réponse correcte : A

Explication : Les accolades permettent de regrouper plusieurs imports partageant le même préfixe de namespace.

9. Comment déclare-t-on correctement un sous-namespace nommé Database, situé sous le namespace principal MonApplication ?

A. namespace MonApplication.Database;
B. namespace MonApplication/Database;
C. subnamespace MonApplication\Database;
D. namespace MonApplication\Database;

Réponse correcte : D

Explication : En PHP, l'antislash \ sert à séparer les niveaux d'un namespace. La déclaration correcte est donc :

namespace MonApplication\Database;

Ce namespace correspond à Database, sous le namespace MonApplication.